

ParseBurger

Especificación Formal de Herramientas

Procesamiento de Lenguaje Natural — Entrega de proyecto

Laura Sofía Peñaloza López

Laura Tatiana Coicue Poquigueue

1. Descripción general del sistema

ParseBurger es un sistema de PLN para pedidos de hamburguesas que combina cuatro formalismos en una arquitectura de capas secuenciales: (1) **gramática regular** para normalización léxica, (2) **CFG** para análisis sintáctico, (3) **DCG** con unificación de rasgos morfológicos para verificación semántica, y (4) **ATN** para gestión del diálogo. Cada capa consume la salida de la anterior y la enriquece antes de pasarla a la siguiente.

2. Autómata de Transiciones de Red (ATN)

2.1 Definición formal — 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$

Componente	Notación	Cardinalidad	Descripción	Valor
Estados	Q	5 estados	Nodos del ATN	INICIO, ESCUCHANDO, AMBIGUO, CONFIRMANDO, FIN
Alfabeto	Σ	texto libre	Mensajes del usuario normalizados	Cadenas de tokens del pedido
Transiciones	δ	tabla	Función $Q \times \Sigma \rightarrow Q$	Ver tabla §2.2 (DIALOGO_ATN)
Estado inicial	q_0	1	Estado de arranque	INICIO
Estados finales	F	1	Pedido completado	{FIN}

Cuadro 2: 5-tupla del ATN implementado en `atn_mesero.py`

2.2 Función de transición δ — tabla DIALOGO_ATN

Las subredes (`_subred_inicio`, `_subred_escuchando`, `_subred_ambiguo`, `_subred_confirmando`) analizan la entrada y devuelven un arco. El motor de la clase `ATNMesero` consulta `DIALOGO_ATN` con ese arco y actualiza el estado, separando la lógica de cada estado de la función de transición.

3. Gramática Libre de Contexto (CFG)

3.1 Definición formal $G = (V, T, P, S)$

V S, PEDIDO, NP_LIST, NP_PEDIDO, VERBO_PEDIR, CANTIDAD, PRODUCTO, TIPO_BURGER, ADJETIVO_BURGER, MOD_LIST, MOD_LIST_TAIL, MOD, NEG, POS, EXTRA, INTENSIF, CONJ, INGREDIENTE, ENTRADA_INVALIDA, ITEM_INVALIDO

T quiero, dame, quisiera, ponme, un, una, dos, tres, cuatro, cinco, hamburguesa(s), clasica(s), doble(s), especial(es), vegana(s),

Estado origen	Arco (condición)	Estado destino	Acción
INICIO	inicio	ESCUCHANDO	Muestra menú
ESCUCHANDO	pedido_valido	CONFIRMANDO	Registra pedido DCG
ESCUCHANDO	pedido_ambiguo	AMBIGUO	Solicita A/B
ESCUCHANDO	entrada_invalida	ESCUCHANDO	Rechaza ítem
ESCUCHANDO	error_concordancia	ESCUCHANDO	Pide reformulación
ESCUCHANDO	no_reconocido	ESCUCHANDO	Pide reformulación
AMBIGUO	resolucion_a	CONFIRMANDO	Aplica interp. A
AMBIGUO	resolucion_b	CONFIRMANDO	Aplica interp. B
AMBIGUO	invalido	AMBIGUO	Repite pregunta
CONFIRMANDO	confirmar	FIN	Cierra pedido
CONFIRMANDO	cancelar	ESCUCHANDO	Descarta pedido

Cuadro 3: Arcos del ATN. Cada subred devuelve un arco; el motor actualiza el estado.

sin, con, extra, mucho, bastante, poco, y, pero,
 cebolla, queso, tomate, lechuga, pepinillo, mayonesa, mostaza,
 ketchup, carne, tocino, salsa, perro_caliente, pizza, hotdog, tacos

S S (símbolo inicial)

P Ver tabla 4

3.2 Producciones

Cuadro 4: Producciones P de la CFG (implementadas en `gramatica.py`)

No terminal	Producción(es)
S	$\rightarrow \text{PEDIDO} \mid \text{ENTRADA_INVALIDA}$
PEDIDO	$\rightarrow \text{VERBO_PEDIR NP_LIST}$
NP_LIST	$\rightarrow \text{NP_PEDIDO CONJ NP_LIST} \mid \text{NP_PEDIDO}$
NP_PEDIDO	$\rightarrow \text{CANTIDAD PRODUCTO MOD_LIST} \mid \text{CANTIDAD PRODUCTO}$
VERBO_PEDIR	$\rightarrow \text{quiero} \mid \text{dame} \mid \text{quisiera} \mid \text{ponme}$
CANTIDAD	$\rightarrow \text{un} \mid \text{una} \mid \text{dos} \mid \text{tres} \mid \text{cuatro} \mid \text{cinco}$
PRODUCTO	$\rightarrow \text{TIPO_BURGER} \mid \text{hamburguesa ADJETIVO_BURGER} \mid \text{hamburguesas ADJETIVO_BURGER}$
TIPO_BURGER	$\rightarrow \text{hamburguesa} \mid \text{clasica} \mid \text{doble} \mid \text{especial} \mid \text{vegana (+ plurales)}$
ADJETIVO_BURGER	$\rightarrow \text{clasica} \mid \text{doble} \mid \text{especial} \mid \text{vegana (+ plurales)}$
MOD_LIST	$\rightarrow \text{MOD MOD_LIST_TAIL} \mid \text{MOD}$
MOD_LIST_TAIL	$\rightarrow \text{CONJ MOD MOD_LIST_TAIL} \mid \text{pero MOD MOD_LIST_TAIL} \mid \text{CONJ MOD} \mid \text{pero MOD} \mid \text{CONJ INGREDIENTE MOD_LIST_TAIL} \mid \text{CONJ INGREDIENTE} \mid \text{MOD MOD_LIST_TAIL} \mid \text{MOD}$ <i>(ambigüedad intencional en $\text{sin } X \text{ y } Y$, resuelta por PCFG)</i>
MOD	$\rightarrow \text{NEG INGREDIENTE} \mid \text{POS INGREDIENTE} \mid \text{EXTRA INGREDIENTE} \mid \text{POS EXTRA INGREDIENTE} \mid \text{POS INTENSIF INGREDIENTE}$
NEG	$\rightarrow \text{sin}$
POS	$\rightarrow \text{con}$
EXTRA	$\rightarrow \text{extra} \mid \text{doble}$

continúa...

No terminal	Producción(es)
INTENSIF	→ mucho bastante poco
CONJ	→ y
INGREDIENTE	→ cebolla queso tomate lechuga pepinillo mayonesa mostaza ketchup carne tocino salsa
ENTRADA_INVALIDA	→ VERBO_PEDIR CANTIDAD ITEM_INVALIDO VERBO_PEDIR ITEM_INVALIDO
ITEM_INVALIDO	→ perro_caliente pizza hotdog tacos

4. Gramática de Cláusulas Definidas (DCG) y rasgos morfológicos

El módulo `lexico.py` asocia a cada token un DAG de rasgos con cuatro atributos: **cat** (categoría sintáctica), **num** (sg|pl), **gen** (masc|fem|neu) y **pers** (solo en verbos). La función `unificar(dag1, dag2)` implementa unificación estándar: retorna el DAG fusionado o `None` ante conflicto de rasgos. La tabla `CONCORDANCIA_GEN` especifica las combinaciones (`gen_cantidad`, `gen_producto`) válidas; el género `neu` actúa como comodín.

gen(cantidad)	gen(producto)	Válido	Ejemplo
masc	masc	✓	<i>un doble</i>
fem	fem	✓	<i>una clásica</i>
neu	fem	✓	<i>dos hamburguesas</i>
neu	masc	✓	<i>dos dobles</i>
neu	neu	✓	<i>dos especiales</i>
masc	neu	✓	<i>un especial</i>
fem	neu	✓	<i>una especial</i>
masc	fem	×	<i>*un hamburguesa</i>
fem	masc	×	<i>*una doble (si masc)</i>

Cuadro 5: Tabla `CONCORDANCIA_GEN` implementada en `lexico.py`

5. Integración de las herramientas en el sistema

Las herramientas operan en una tubería (*pipeline*) estrictamente secuencial. La tabla siguiente resume el rol de cada módulo y sus interfaces:

Módulo	Formalismo	Entrada	Salida / Rol
<code>tokenizador.py</code>	Gram. regular	Texto libre del usuario	Tokens normalizados
<code>parser.py</code>	CFG + rec. desc.	Tokens + gramática	Árbol; detecta PEDIDO vs ENTRADA_INVALIDA
<code>ambigüedad.py</code>	PCFG	Lista de árboles CFG	Árbol más probable; detecta sin X y Y
<code>lexico.py</code>	DCG + unificación	Tokens + léxico	Verifica concordancia; dict semántico
<code>atn_mesero.py</code>	ATN de diálogo	Turno + estado actual	Respuesta del mesero + transición de estado

Cuadro 7: Pipeline de módulos y herramientas formales

Flujo para la entrada «quiero una doble sin tomate»:

1. **Tokenizador** normaliza y produce ["quiero","üna","doble","sin","tomate"].
2. **Parser CFG** construye $S \rightarrow \text{PEDIDO} \rightarrow \text{VERBO_PEDIR NP_PEDIDO} \rightarrow \dots$ verificando pertenencia al lenguaje. Si fuese ENTRADA_INVALIDA, el ATN dispara el arco correspondiente sin continuar.
3. **Módulo de ambigüedad** busca el patrón sin <ING>y <ING>; al no hallarlo, cede el control. Con varios árboles CFG, `mejor_arbol_pcfg()` seleccionaría el más probable.
4. **DCG** verifica que *una* (fem,sg) concuerda con *doble* (neu,sg) (fem+neu ✓) y construye {cantidad:üna", producto:"doble", modificadores:[{NEG,"tomate"}]}.
5. **ATN**: la subred ESCUCHANDO recibe el dict semántico, dispara `pedido_valido` y el motor transiciona a CONFIRMANDO.

Puntos de integración críticos:

- **CFG → Ambigüedad**: `parse_todos()` devuelve todos los árboles; si hay más de uno, `mejor_arbol_pcfg()` resuelve la ambigüedad estructural.
- **Ambigüedad → ATN**: el patrón sin X y Y interrumpe el flujo y redirige a la subred AMBIGUO, que espera confirmación del usuario (A/B) antes de construir el dict semántico.
- **DCG → ATN**: `dcg_parse_lista_pedidos()` produce el dict semántico que la ATN almacena en `pedido_actual` y usa en CONFIRMANDO para el resumen del pedido.

Los formalismos descritos están implementados directamente en los módulos del proyecto: la gramática CFG es `grammar` en `gramatica.py`; la DCG y el léxico con rasgos son `lexico.py`; el ATN es la clase `ATNMesero` en `atn_mesero.py` con su tabla `DIALOGO_ATN`.